



**PIMPRI CHINCHWAD EDUCATION TRUST'S
PIMPRI CHINCHWAD COLLEGE OF
ENGINEERING**

Department of Computer Science and Engineering (AI & ML)

A Project Report On

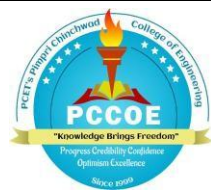
Swachh Sena : Garbage Reporting System

Submitted By

Name	Prn No
Pushkar Bhoge	125B2C006
Pranav Khade	125B2C007
Pushkaraj Deshpande	125B2C011

Under the Guidance of Project Guide

Mrs. Sarika Bartakke



Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
Department of Computer Science and Engineering (AI & ML)

Project Report

❖ Abstract

Swachh Sena is a web-based garbage reporting and management system designed to improve urban cleanliness and waste handling. It provides a simple platform where citizens can report garbage issues by uploading images along with location details, creating a direct communication channel between the public and municipal authorities. The system allows administrators to view complaints, assign cleaning teams, and track the status of each report through stages like pending, assigned, and cleaned, ensuring efficient management.

The application also includes features such as before-and-after images for transparency, an AI chatbot for assistance, a donation system, blogs, and multilingual support to enhance user engagement. Built using modern technologies like Node.js, Express.js, MongoDB, and services like Cloudinary and Stripe, the system follows the MVC architecture for better scalability and organization. Overall, Swachh Sena offers a reliable and user-friendly solution that promotes community participation and cleaner cities.

❖ List of Figure, Table, Abbreviation

Category	Number / Name	Description
Figures	Fig 1 – High Level Design (HLD)	Overview of system structure and major components
	Fig 2 – Low Level Design (LLD)	Detailed internal design and module logic
	Fig 3 – Use Case Diagram	Shows interactions between users and system
	Fig 4 – System Architecture	Overall system components and their connections
	Fig 5 – MVC Architecture	Structure separating Model, View, and Controller
	Fig 6 – ER Diagram	Database entities and their relationships
	Fig 7 – DFD Level 0	Basic system data flow overview
	Fig 8 – DFD Level 1	Detailed data flow within system processes
	Fig 9 – Home Page	Main user interface of the system
	Fig 10 – Admin Dashboard	Interface for admin control and management
	Fig 11 – Report Garbage Page	Page for submitting garbage reports
	Fig 12 – View Reports Page	Page to view submitted reports.
Tables	Table 1 – Test Case Report	List of test cases and results
	Table 2 – Bug Report Log	Record of identified bugs and fixes
Abbreviations	MVC	Model View Controller
	ER	Entity Relationship
	DFD	Data Flow Diagram
	API	Application Programming Interface
	DB	Database
	UI	User Interface
	UX	User Experience
	AI	Artificial Intelligence
	HTTP	HyperText Transfer Protocol
	HTTPS	HyperText Transfer Protocol Secure
	SRS	Software Requirement Specification
	HLD	High Level Design
	LLD	Low Level Design
	SDD	Software Design Description

Chapter 1: Introduction

1.1 Motivation

The main motivation behind developing this project is to create a cleaner and healthier environment by encouraging citizen participation in waste management. Many people are willing to report issues but lack a proper platform to do so. This project aims to empower citizens by giving them a simple and effective tool to raise their concerns.

Another important motivation is to improve accountability and transparency in municipal services. By introducing features like status tracking and before/after images, the system ensures that reported issues are resolved properly and can be verified by users.

Additionally, the project is motivated by the need to use modern technology to solve real-world problems. By integrating features like AI chatbot support, online donations, and multilingual access, the system becomes more interactive, inclusive, and user-friendly.

Overall, Swachh Sena is driven by the goal of making waste management more efficient, transparent, and accessible while contributing to the vision of cleaner and smarter cities.

1.2 Background

In today's rapidly growing urban environment, waste management has become a major challenge for cities. Improper disposal of garbage not only affects the cleanliness of public spaces but also leads to environmental pollution and health-related issues. Despite the presence of municipal systems, there is often a lack of efficient communication between citizens and authorities when it comes to reporting garbage problems.

Most existing systems do not provide a simple and accessible way for citizens to report such issues. As a result, garbage remains unattended for long periods, leading to unhygienic conditions. There is also a lack of transparency in the cleaning process, as citizens are not informed about the status of their complaints.

The Swachh Sena system is developed to address these challenges by providing a digital platform where users can report garbage issues easily using images and location details. It helps authorities manage complaints efficiently and ensures proper tracking from reporting to resolution.

CHAPTER 2: PLANNING

2.1 Statement of Work (SOW)

The main objective of the Swachh Sena project is to design and develop a web-based system that allows citizens to report garbage issues and enables authorities to manage and resolve those complaints efficiently.

The system focuses on:

- Providing a user-friendly platform for reporting garbage
- Enabling administrators to monitor and manage complaints
- Ensuring transparency through status tracking and image verification
- Encouraging community participation through additional features like blogs and donations

2.2 Resource Planning

Resource planning involves identifying the resources required for the successful development of the system.

Human Resources

- Developers for backend and frontend development
- Admin for managing reports and system monitoring

Technical Resources

- Programming: Node.js, Express.js
- Database: MongoDB
- Frontend: EJS
- Tools: Cloudinary (image storage), Stripe (payments)

Platform Resources

- Deployment platform: Render
- Database hosting: MongoDB Atlas

2.3 Risk Management Plan

Every project involves certain risks that need to be identified and managed properly.

Technical Risks

- System errors or bugs → handled through testing and debugging
- Data loss → prevented using cloud database backups

Security Risks

- Unauthorized access → handled using authentication and session management
- Data privacy issues → secured using encryption and HTTPS

Operational Risks

- Server downtime → managed using reliable cloud hosting
- Payment failures → handled through secure Stripe integration

Scalability Risks

- Increase in users and data → handled using scalable cloud services

2.4 Feasibility Study**Technical Feasibility**

The project is technically feasible as it uses widely available technologies like Node.js and MongoDB, which are easy to implement and maintain.

Economic Feasibility

The project is cost-effective as it uses free or low-cost cloud services and open-source technologies.

Operational Feasibility

The system is user-friendly and can be easily used by both citizens and administrators without requiring special training.

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Software Requirement Specification (SRS)

The Software Requirement Specification describes both functional and non-functional requirements of the system.

Functional Requirements

The system should provide a secure user authentication mechanism that allows users to register and log in efficiently. Once authenticated, users should be able to report garbage issues by uploading images along with accurate location details, enabling proper identification of problem areas. The admin should have access to a centralized dashboard where all reported issues can be viewed and managed effectively. Additionally, the admin must be able to assign cleaning teams to specific reports and update their status as Pending, Assigned, or Cleaned, ensuring transparency and proper workflow management. Users should also have the ability to track the status of their submitted reports in real time. Furthermore, the system should include a donation feature that allows users to contribute financially toward cleanliness initiatives. To enhance user interaction and support, the platform should also integrate an AI chatbot that can assist users with queries and provide relevant information.

Non-Functional Requirements

The system should ensure a high level of security to protect user data, including personal information and uploaded content, by implementing proper authentication, authorization, and data encryption techniques. It should be fast and responsive, allowing users to perform actions such as reporting issues, uploading images, and viewing updates without noticeable delays. The system must also be scalable so that it can efficiently handle a growing number of users and increased data without affecting performance. Additionally, the interface should be user-friendly and easy to navigate, enabling users of all technical backgrounds to interact with the system comfortably and complete tasks without confusion.

3.2 High-Level Design (HLD)

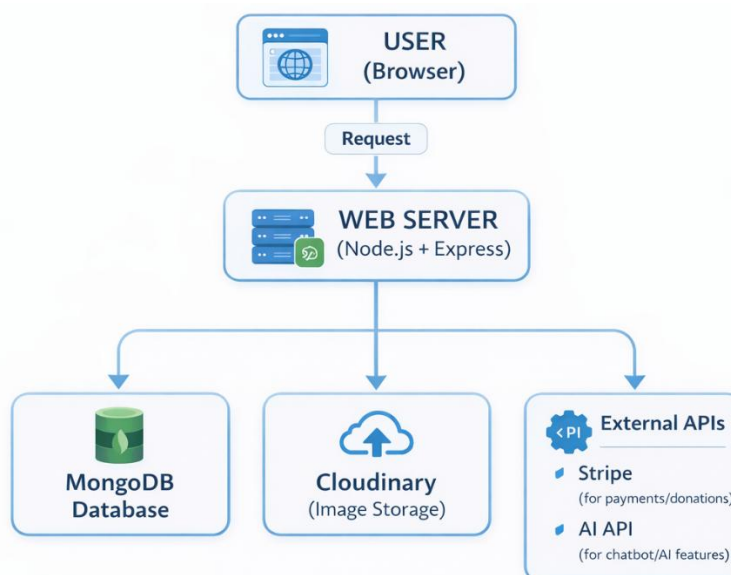


Fig. 1. High Level Design (HLD)

3.3 Low-Level Design (LLD)

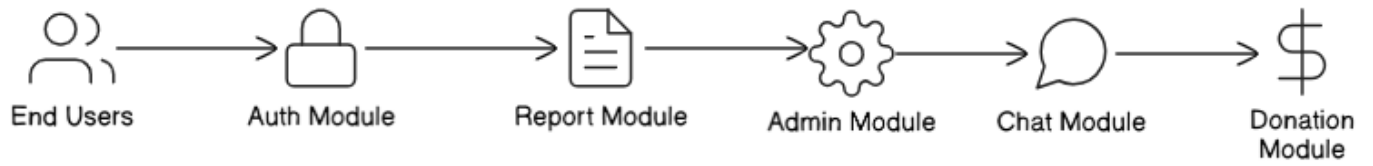


Fig. 2. Low Level Design (LLD)

3.4 Use Case Analysis

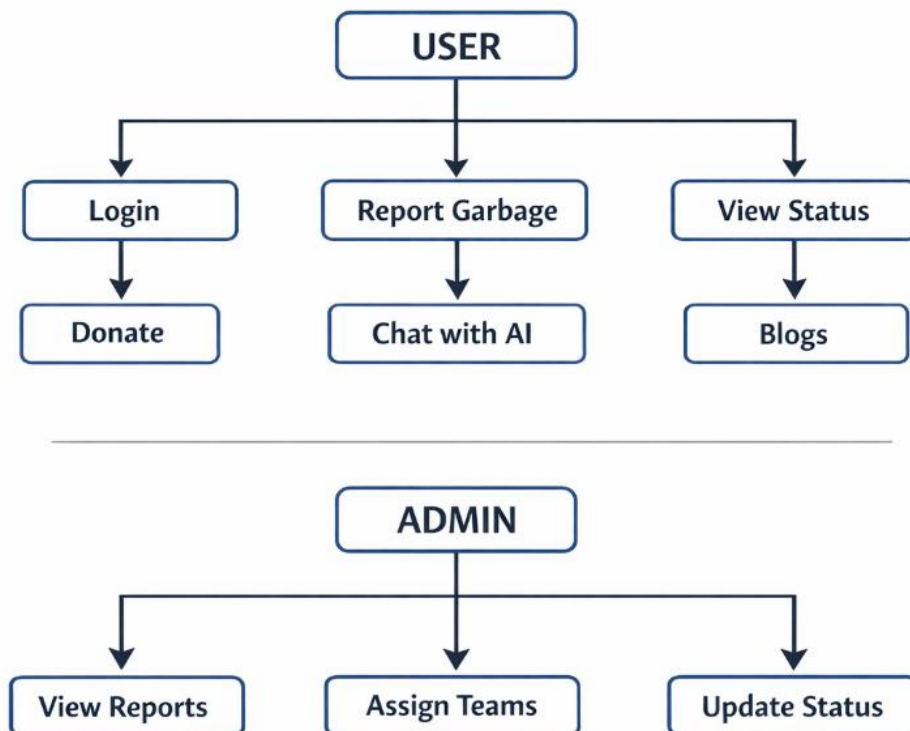


Fig. 3. Use Case Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The Swachh Sena system follows a client-server architecture, which allows users to interact with the system through a web browser while the backend server handles all processing and data management.

When a user submits a garbage report, the request is sent to the server, where it is processed and stored in the database. Images are uploaded to Cloudinary, and other services like Stripe and AI APIs are used for payments and chatbot functionality. This separation ensures better performance and scalability.

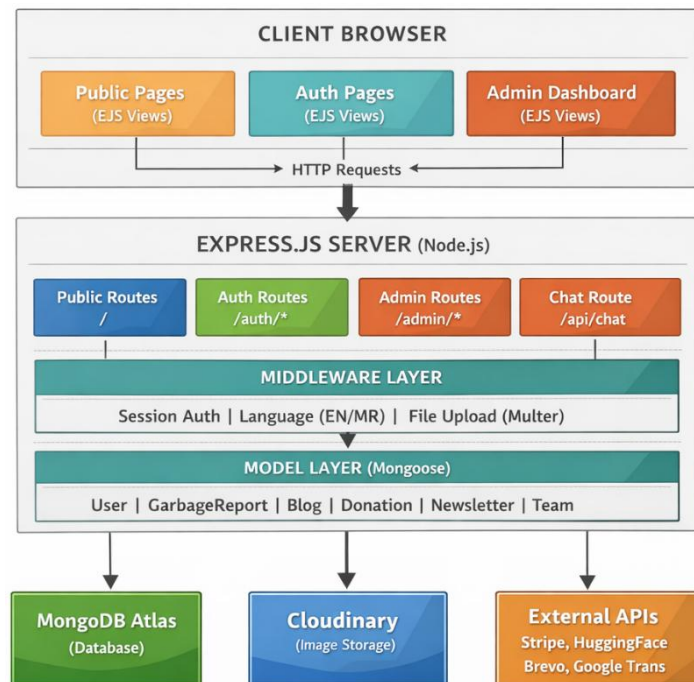


Fig. 4. System Architecture

4.2 MVC Architecture

The system uses the MVC (Model-View-Controller) architecture to organize the code and separate different functionalities.

- The View represents the user interface, where users interact with the system.
- The Controller handles user requests, processes logic, and communicates with the model.
- The Model interacts with the database and manages data storage.

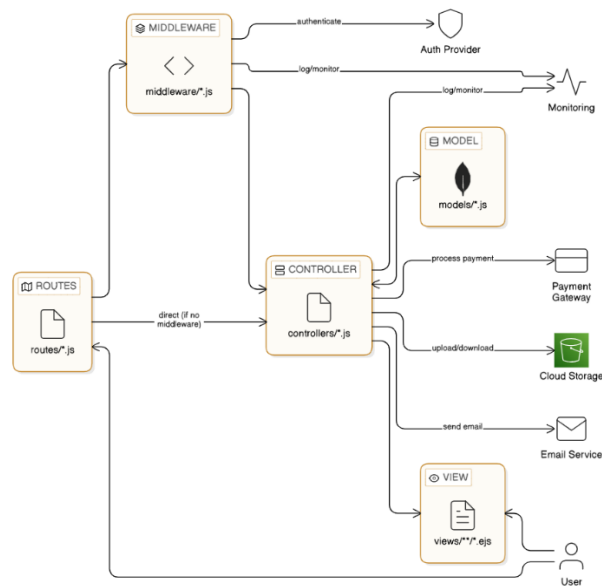


Fig. 5. MVC Architecture

4.3 ER Diagram (Database Design)

The ER diagram represents how different entities in the system are related to each other. In Swachh Sena, the main entity is the Garbage Report, which is connected to users and teams.

Each user can create multiple reports, and each report can be assigned to a cleaning team. Additional entities like blogs, donations, and chat messages are also linked to users. This structure helps in organizing data efficiently and maintaining relationships between different components.

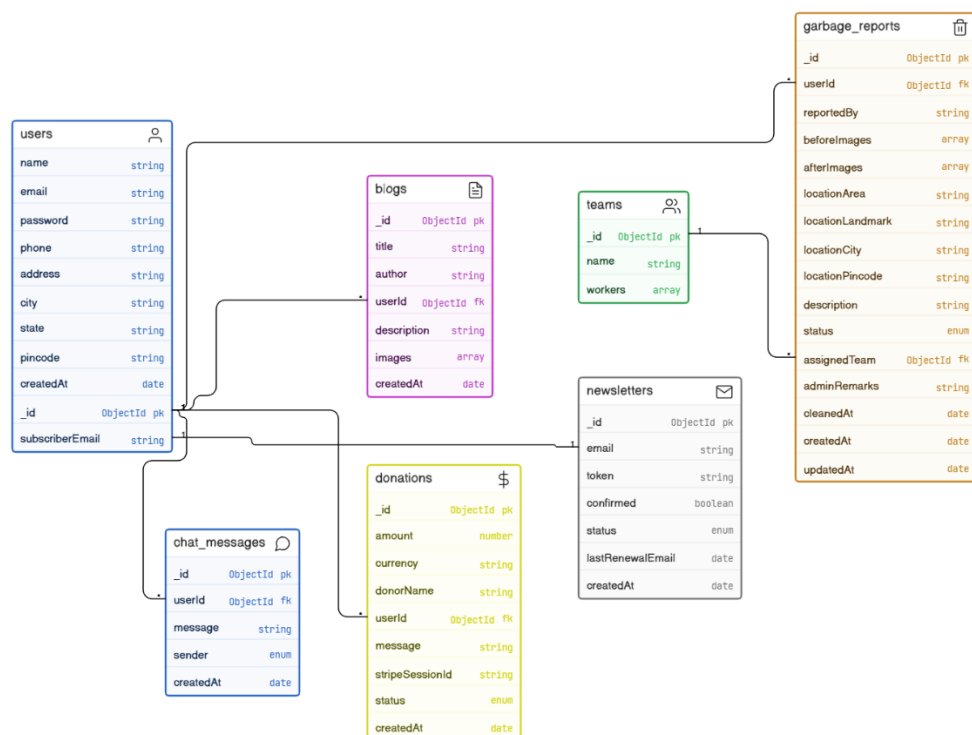


Fig. 6. ER Diagram

4.4 Data Flow Diagram (DFD)

The Data Flow Diagram shows how data moves within the system and how different processes interact with each other.

DFD Level 0:

This diagram provides a high-level view of the system, showing the interaction between the user, system, and admin.

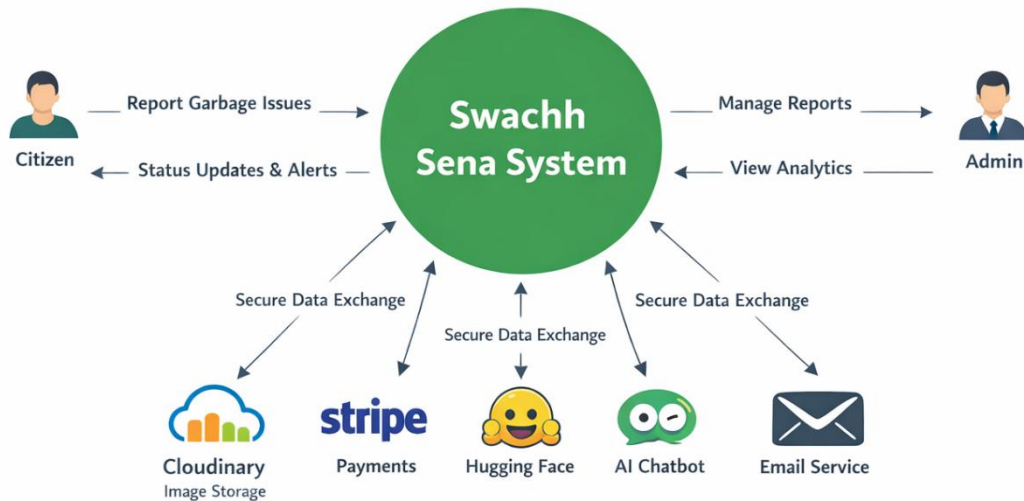


Fig. 7. DFD Level 0

DFD Level 1 :

This diagram shows the detailed internal processes of the system. It includes authentication, report submission, blog creation, donations, and chatbot interaction.

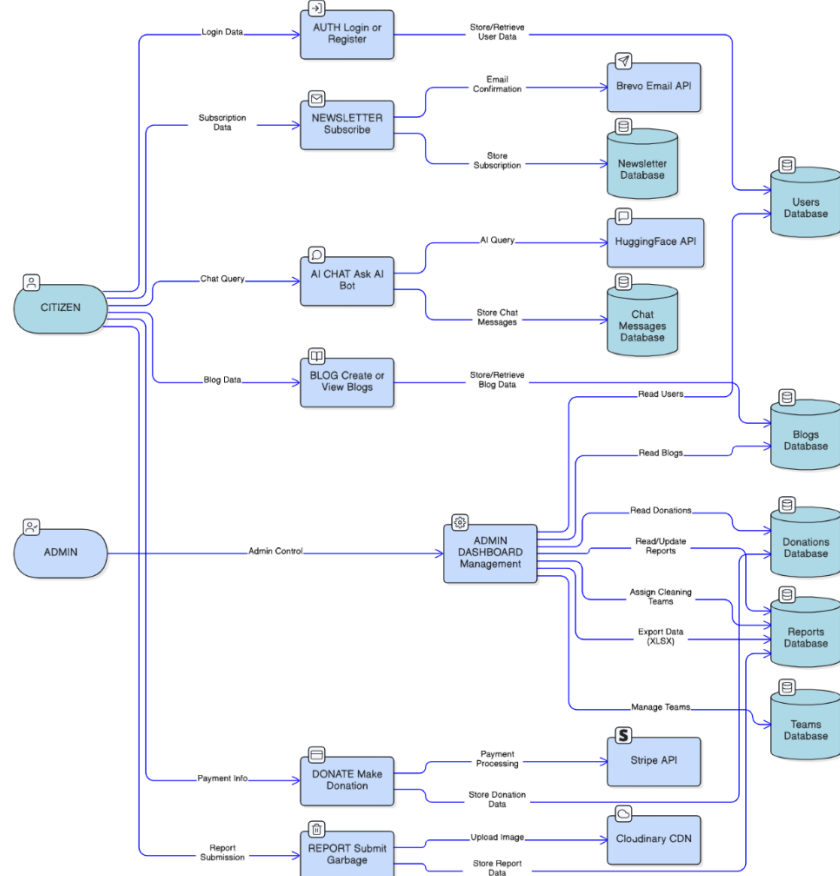


Fig. 8. DFD Level 1

CHAPTER 5: IMPLEMENTATION

5.1 Coding Approach

The system is developed using JavaScript with Node.js and Express.js for backend development. The coding follows a modular approach, where different functionalities such as authentication, reporting, admin management, and chatbot are implemented in separate modules.

The project follows the MVC architecture, which helps in organizing the code into models, views, and controllers. This makes the code easier to understand, maintain, and update. Each module is designed to perform a specific function, reducing complexity and improving efficiency.

5.2 Code Comments

Code comments are added throughout the project to explain important parts of the code. These comments help developers understand the logic and functionality of different sections.

For example, comments are used to:

- Explain complex logic in controllers
- Describe the purpose of functions and variables
- Clarify API request handling
- Indicate important validations and conditions

5.3 API Documentation

API documentation is created to describe how different parts of the system communicate with each other. It provides details about available endpoints, request methods, parameters, and responses.

Some important APIs in the system include:

- /auth → Handles user registration and login
- /report → Used for submitting garbage reports
- /admin → Used by admin to manage reports
- /api/chat → Handles AI chatbot interaction

5.4 Code Review Records

During the development of the Swachh Sena system, code reviews were conducted regularly to maintain code quality and consistency. Each module was carefully examined to ensure that the logic was correct, the code was readable, and proper coding standards were followed. The review process helped in identifying errors early and improving the overall structure of the application.

Special focus was given to security and performance aspects, such as validating user inputs, managing sessions securely, and optimizing database operations. Any issues found during the reviews were documented and resolved promptly, which helped in making the system more stable and reliable. These reviews ensured that the final application is well-structured, secure, and suitable for real-world deployment.

CHAPTER 6: RESULTS

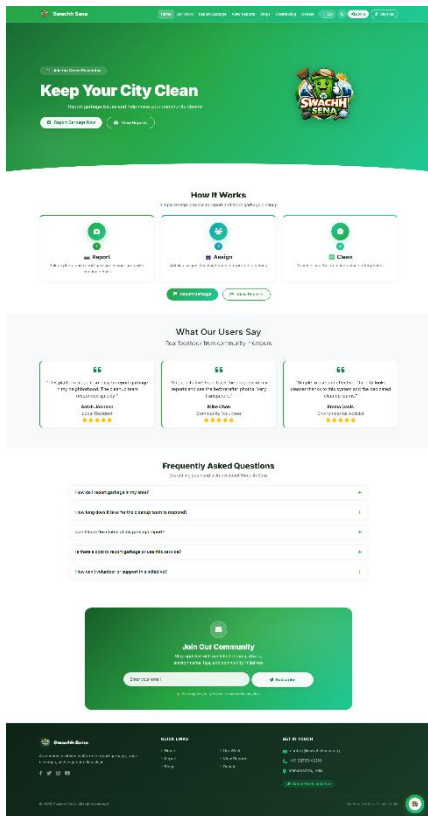


Fig. 9. Home Page

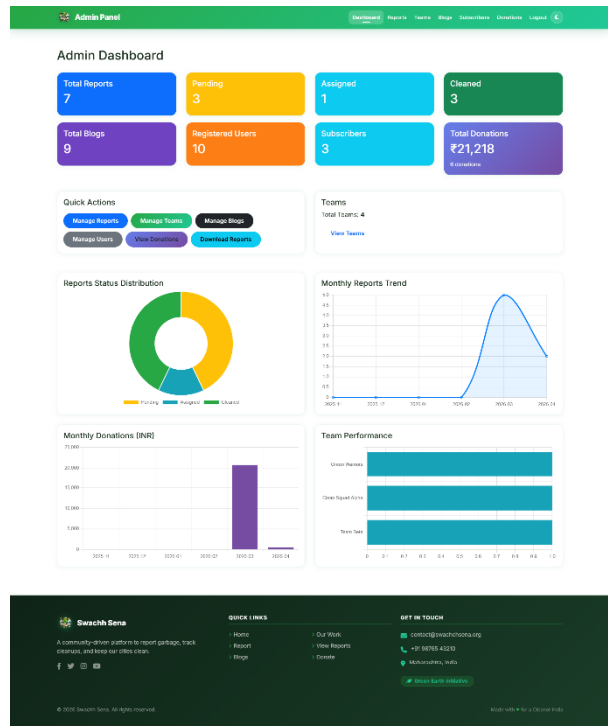


Fig. 10. Admin Dashboard

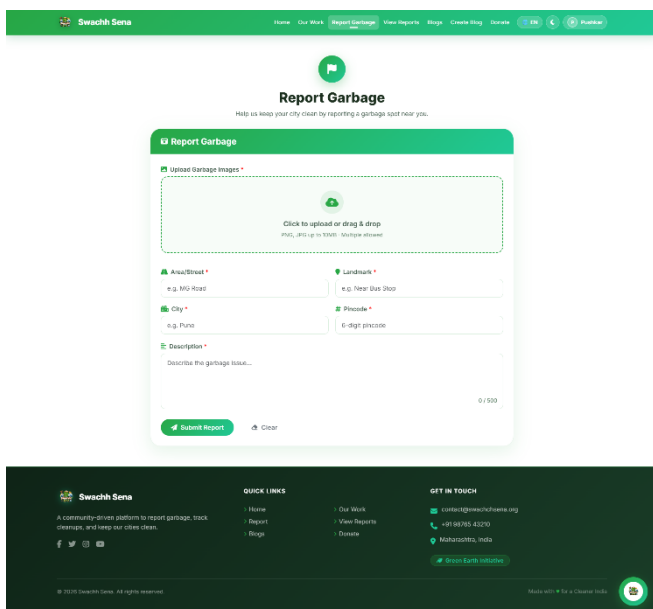


Fig. 11. Report Garbage Page

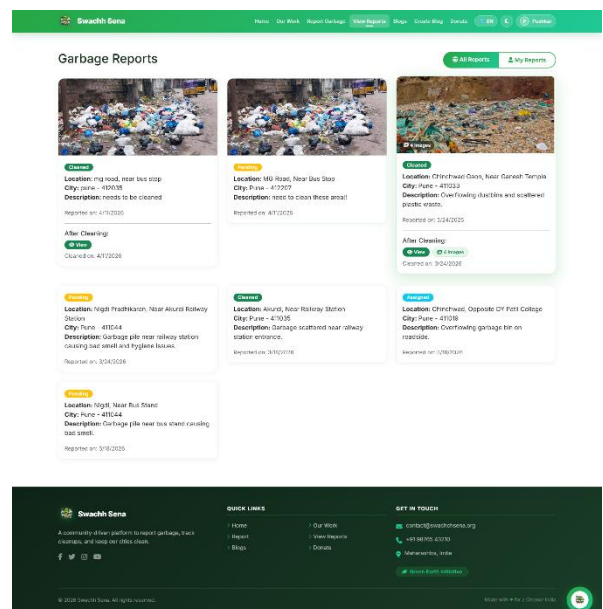


Fig. 12. View Reports Page

CHAPTER 7: TESTING

6.1 Test Plan

A structured test plan was followed to ensure that every part of the system was tested thoroughly. The testing process included checking individual modules as well as the complete system. Functional testing was performed to verify that each feature works as expected, while integration testing ensured that different modules interact smoothly with each other. The system was also tested under different scenarios to confirm its stability and performance.

6.2 Test Cases

The following test cases were designed and executed to validate the system functionality:

TABLE I. Test Case Report

Test Case ID	Test Case Description	Input	Expected Output	Result
TC-01	User Registration	Valid user details	Account created successfully	Pass
TC-02	User Login (Valid)	Correct username & password	User logged in successfully	Pass
TC-03	User Login (Invalid)	Wrong credentials	Error message displayed	Pass
TC-04	Submit Garbage Report	Image + location details	Report stored successfully	Pass
TC-05	View Report Status	Existing report	Status displayed correctly	Pass
TC-06	Admin Login	Valid admin credentials	Admin dashboard opens	Pass
TC-07	View All Reports	Admin request	All reports displayed	Pass
TC-08	Assign Team	Select report & team	Team assigned successfully	Pass
TC-09	Update Report Status	Change to "Cleaned"	Status updated correctly	Pass
TC-10	AI Chatbot Interaction	User query	Response generated by AI	Pass
TC-11	Make Donation	Payment details	Payment processed successfully	Pass
TC-12	Blog Creation	Blog content	Blog published successfully	Pass

6.3 Bug Reports and Test Results

TABLE II. Bug Report Log

Bug ID	Module	Issue Description	Cause	Solution Applied	Status
BR-01	Report Module	Image upload failed for large files	File size limit not handled	Added file size validation and compression	Fixed
BR-02	Authentication	Session expired quickly	Improper session configuration	Updated session timeout settings	Fixed
BR-03	Payment Module	Payment not updating in database	Callback handling error	Corrected Stripe webhook logic	Fixed
BR-04	Admin Module	Status not updating properly	Incorrect database query	Fixed update query logic	Fixed
BR-05	Chat Module	Slow response from chatbot	API delay	Added loading handling and retry logic	Improved
BR-06	UI/Frontend	Form submission without validation	Missing input checks	Added frontend validation	Fixed
BR-07	Report Module	Location not saved correctly	Data mapping issue	Corrected schema mapping	Fixed
BR-08	Blog Module	Blog not displaying after creation	Rendering issue	Fixed view rendering logic	Fixed

CHAPTER 8: DEPLOYMENT

7.1 Deployment Environment

The Swachh Sena application is deployed using a cloud-based platform, which provides reliability and scalability. The backend server is hosted on Render, which allows the application to run continuously and handle user requests efficiently. MongoDB Atlas is used as the cloud database to store all system data securely.

Images uploaded by users are stored using Cloudinary, which reduces the load on the main server and improves performance. The payment functionality is handled through Stripe, ensuring secure and trusted transactions. These services work together to provide a stable and efficient deployment environment.

7.2 Deployment Process

The deployment process begins with preparing the application for production. This includes organizing the code, installing required dependencies, and configuring environment variables such as database connection strings and API keys.

Once the setup is complete, the application is deployed on the Render platform. The server is configured to run the application, and the database is connected through MongoDB Atlas. After deployment, the system is tested to ensure that all features such as report submission, admin operations, chatbot interaction, and payment processing are working correctly.

7.3 Release Notes

The initial release of the Swachh Sena system includes core features such as user registration, garbage reporting, admin dashboard, and report tracking. Later updates added advanced features like AI chatbot support, donation system, blog functionality, and multilingual support. These updates improve the usability and functionality of the system.

7.4 User Manual

The system is designed to be simple and easy to use. Users can register or log in to the system, report garbage by uploading images and entering location details, and track the status of their reports. Admin users can log in to the dashboard, view all reports, assign cleaning teams, and update the status after completion.

Clear instructions are provided within the system interface to guide users, making it accessible even for non-technical users.

7.5 Installation Guide

To run the project locally, the user needs to install Node.js and set up the development environment. After downloading the project files, required dependencies are installed using npm. The database connection and API keys are configured using environment variables.

Once the setup is complete, the application can be started using the appropriate command, and it becomes accessible through a web browser. For production use, the application is deployed on a cloud platform like Render.

CHAPTER 9: CONCLUSION

The Swachh Sena project provides an effective solution for improving waste management by creating a direct connection between citizens and municipal authorities. It allows users to report garbage issues easily by uploading images and location details, ensuring that complaints are properly recorded and addressed. The system simplifies the process of reporting and managing waste, making it more efficient and accessible.

Another important aspect of the system is the transparency it offers. Users can track the progress of their complaints and verify the cleaning process through status updates and images. The admin dashboard helps authorities manage reports efficiently, while features like AI chatbot and secure data handling improve user experience. Overall, the system is reliable, scalable, and suitable for real-world implementation.

Future Scope

The Swachh Sena system can be further improved by developing a mobile application, which would make it more convenient for users to report issues from anywhere. Adding GPS-based location detection can help in automatically capturing accurate location details, reducing manual effort and improving reporting accuracy. Real-time notifications can also be introduced to keep users updated about the status of their complaints.

In addition, advanced technologies like AI-based image recognition can be used to automatically identify and classify garbage from uploaded images. Data analytics and heatmaps can help authorities understand waste patterns and take preventive measures. With continuous improvements and integration of new technologies, the system can become more powerful and contribute significantly to smart city development.